Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 12

Lecture – 61

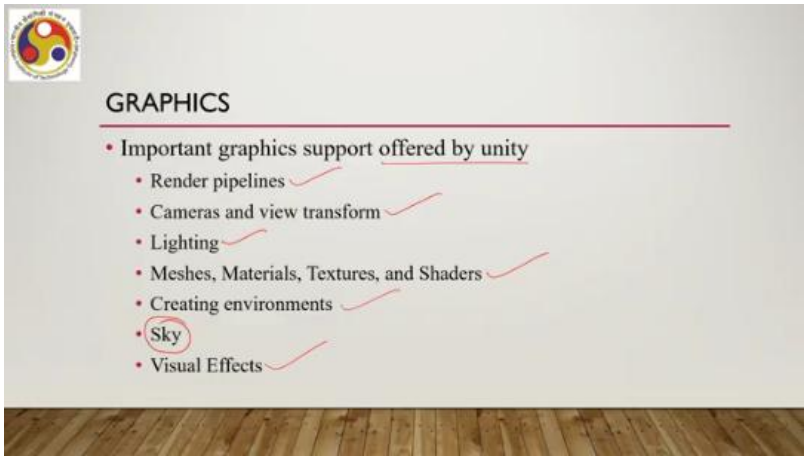
Linking theory to practice

Hello and welcome to the NPTEL MOOCs course on Foundations for Virtual and Augmented Reality Systems. We have started our discussion on linking theory to practice. So, in this part of the lecture, we will continue our discussion. So, here our focus is on linking theory to implementation, and in order to do that, we are talking about an implementation platform, that is the Unity environment, the Unity developer environment. which includes an editor interface through which a developer can create a system, build it, and deploy it.

So, we will continue our discussion on this Unity developer interface and how it provides the support required for building an XR experience. So, our objective was, in the previous part of the lecture, we talked about the objective. As I mentioned, the objective is to get support for graphics pipeline operations so that a developer does not need to worry too much about how to implement the pipeline stages. So, let us see how the Unity developer environment provides that support.

Now, important graphics support that is offered by Unity is render pipeline support. So, Unity provides support to implement the entire pipeline. Camera and view transform support, that is, of course, one stage of the pipeline. Lighting, another stage of the pipeline, is supported.

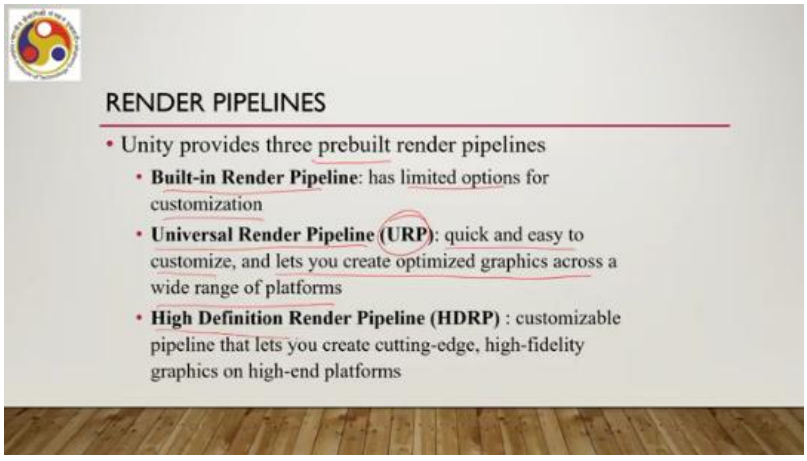
Meshes, materials, textures, and shaders are different components of the pipeline. Creating an environment, that is, a world scene or world coordinate system scene creation, special environments like skies, and different visual effects that you can implement on your system. So, it is not that Unity only provides support for the pipeline, but it also lets the user have customized control over individual stages of the pipeline. So, let us first talk about the pipeline implementation support.



So, Unity provides three pre-built render pipelines. One is the built-in render pipeline. It has limited options for customization. So, whatever is there, you have to work with that. Then, Unity supports the Universal Render Pipeline, or URP.

Now, this is a quick and easy way to customize. This pipeline is quick and easy to customize and lets you create optimized graphics across a wide range of platforms. So, many stages are optimized, or optimization is taken care of by the pipeline implementation itself. So, the developer need not bother about those aspects. Here, you can recollect that we talked about resource optimization, how to optimize the resources.

So, these types of pipeline implementations actually take care of that concern to some extent by taking care of optimization of the pipeline implementation so that resource requirements are minimized. There is also a high definition render pipeline, HDRP. It is also a customizable pipeline that lets you create cutting-edge, high-fidelity graphics on high-end platforms. So this is specifically meant for photorealistic implementation, which requires a good amount of resources.



So it is typically meant for high-end platforms, high-end graphics workstations with good GPU support. A developer can also create a custom render pipeline using Unity's scriptable render pipeline APIs. Now, here in the subsequent part of the lecture, we are going to focus

on the built-in render pipeline and the features provided by this pipeline. So first, let us try to understand how this pipeline supports object definition.

So object definition is the first stage we discussed in our graphics pipeline. So let us see how Unity provides support for that. Now Unity provides object definition support in the form of something called models. These are nothing but files containing data on the shape and appearance of 3D objects.

So, these files are essentially the object representation. Essentially, the object definition is stored in files, which is the first stage of the pipeline we talked about earlier. Now, the model files can contain a variety of data, including mesh, material, and texture. They can also contain animation data for animated characters. If an object represents a character which is animated, then animation-related data can also be stored in the model file.

Unity supports a number of standard and proprietary model file formats. So, you can create models in different formats and still import them to Unity to make them work. Such as .fbx format, .dae format, .dxf format, .obj format, and so on. There are more than 10 such formats supported in Unity. Now, these models contain data related to several things, including mesh, materials, and textures.

So, let us see what those data are. 3D meshes are the main graphics primitives of Unity, and we have seen how we can convert an object definition to a mesh representation. We have discussed it earlier. So, there is direct support for that in Unity. Unity supports triangulated or quadrangulated polygonal meshes.

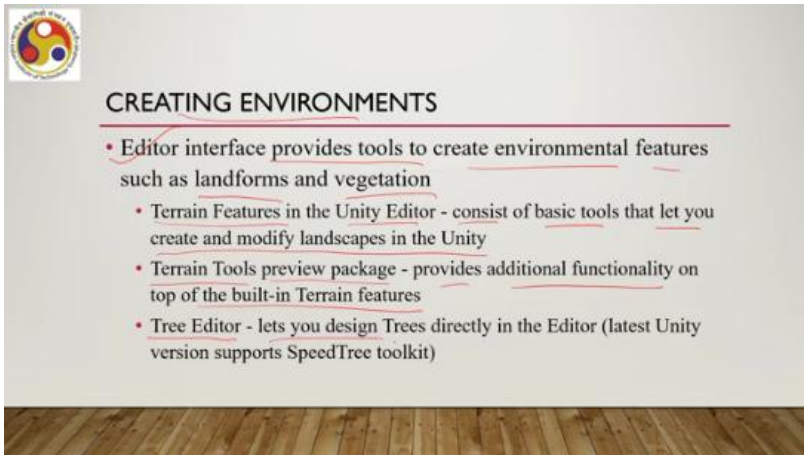
So, either triangles or quadrangles are supported. The mesh renderer takes geometry from the mesh filter and renders it at the position defined by the game object's transform component. This is how the workflow in Unity for rendering meshes works. There is another kind of data stored in the model files, which is materials. So, meshes describe shapes; how shapes

what is the shape of the object, whereas materials describe the appearance of the surface. How a surface should be rendered, including references to the textures used, tiling information, color tints, and more. So, essentially, the appearance of the object is encoded in the form of materials in the model file. Then we have textures. which are bitmap images applied over the mesh surface.

So, essentially, a way to assign colors to the surface. A material may contain references to textures as well. In addition to basic color, textures can represent many other aspects of a material's surface, such as reflectivity and roughness. That is about object representation. Let us see how it supports scene creation, essentially transferring objects from local coordinates to world coordinates.

So, Unity allows us to create an environment, as we have discussed in the previous part. The editor interface provides tools to create environmental features such as landforms and vegetation. Terrain features in the Unity editor can be utilized. It consists of basic tools that let you create and modify the landscape in the Unity environment. Essentially, you can manipulate the landscape in your scene.

There are also terrain tools, the Terrain Tools Preview package, which provides additional functionality on top of the built-in terrain features. And we can also have Also, make use of the Tree Editor, which lets you design trees directly in the editor. Now, the latest Unity version supports the SpeedTree toolkit instead of the Tree Editor. However, for previous earlier versions, the Tree Editor is still there.

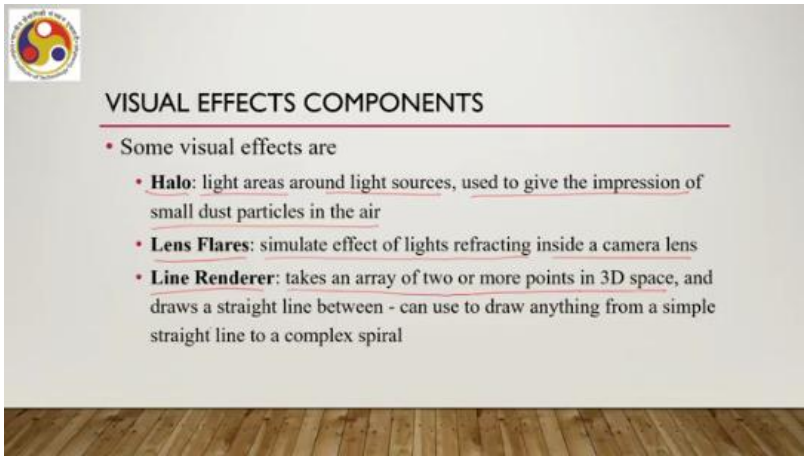


Then there is a special type of scene that is the sky, a type of background that a camera draws before it renders a frame. A developer can use a skybox to render a sky. Now, a skybox is nothing but a cube with different textures on its faces so that you get the feel of having a 360-degree environment where, whenever you move your head, you get to see something different. So, Unity

The way Unity renders the skybox, the sky always renders at the back. Then, let us have a look at the lighting stage support. So, how Unity supports different lighting calculations. Unity provides support for realistic lighting for game objects. Here again, I would like to emphasize once more that when I use the term Unity, I am not referring to the Unity 3D game engine, which is a middleware.

I am referring to the Unity developers environment. So, Unity provides support for realistic lighting for the game objects, and we can specify the intensity, direction, and color of the light that falls on the game object. We can even do more: render real-time shadows with shadow mapping. Now, there are several visual effects that can be applied to the camera, game objects, light sources, and other elements of the game.

Some of those visual effects are, say, the halo effect, where light areas around light sources can be created. Typically, such effects are used to give the impression of small dust particles in the air. Then we can have the lens flare effect. This simulates the effect of lights refracting inside a camera lens and the line renderer effect, which takes an array of two or more points in 3D space and draws a straight line between



those points, which we can use to draw anything from a simple straight line to a complex spiral. Another important stage in the 3D graphics pipeline is the view transformation or viewing pipeline. So Unity supports the taking a snapshot analogy that we have discussed while talking about the 3D graphics pipeline. Now, this concept is supported by the Unity interface, the Unity development environment. So, this is done by the camera concept.

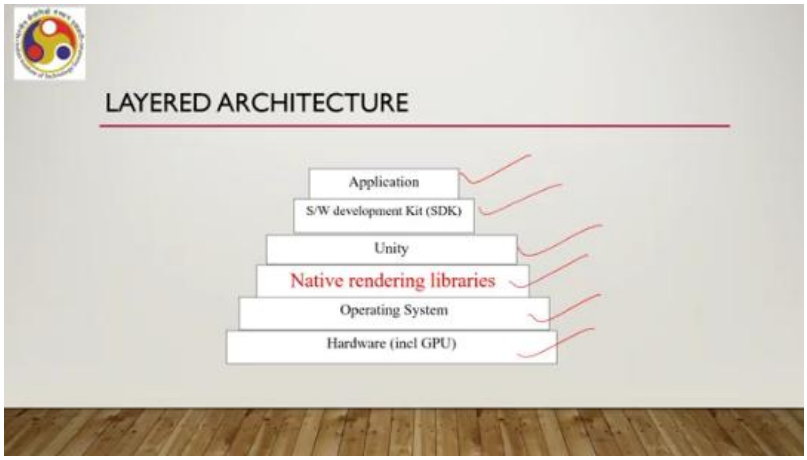
So, it allows us to set up a camera, and then accordingly it does other calculations on its own. The developer does not need to do anything. Now, a camera can be created by adding a camera component to a game object; that is the standard way of performing operations in Unity. Now, finally, let us see what kind of libraries are supported by Unity. So, of course, I said that the developer does not need to bother about the graphics pipeline implementation.

Now, the question is whether Unity itself implements the pipeline. In fact, Unity does not implement the pipeline itself. Instead, it relies on libraries, graphics libraries. It takes recourse to library functions to implement the pipeline. And several library functions are supported.

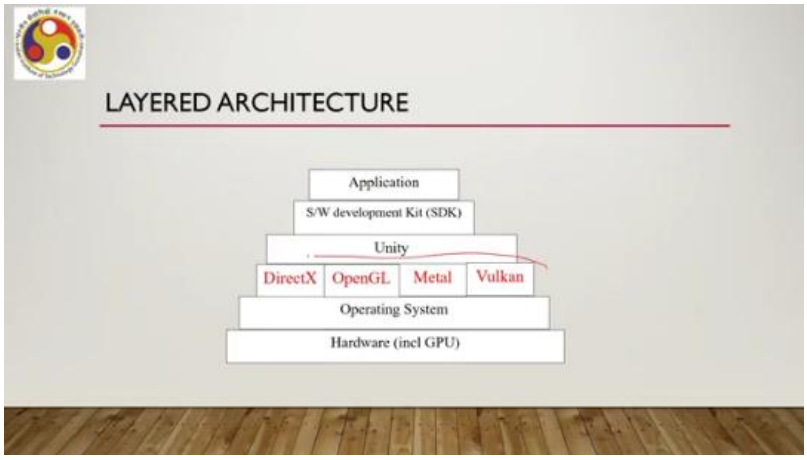
Several libraries are supported by Unity. Let us have a look at what kind of library support is there. Now, in order to do that, let us... recollect our layered architecture of an XR system. So, at the top we have the application layer where the developers are building the application.

Then we have software development kits or SDKs which provide specific library APIs through which the developer can enhance the experience. Then we have the middleware game engine. So, in our case it is Unity. Unity takes help of native rendering libraries to render or to implement the graphics pipeline.

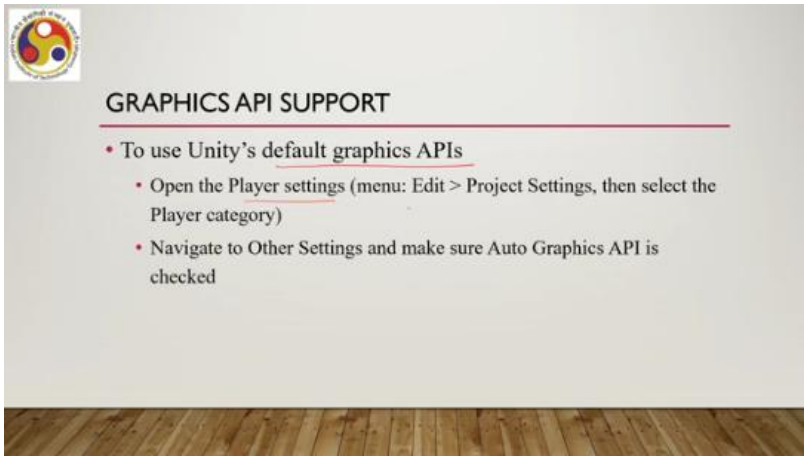
Below it, the operating system and below that the hardware including the GPU. So, in this layered architecture, the native library support or native libraries fall below the Unity layer. And what are the libraries that are supported by Unity? DirectX, which is primarily in Windows environment, Metal, Apple systems, OpenGL, Open Library, Open Graphics Library and Vulkan.



These libraries are supported. Along with that, many more libraries are also supported, but these are the major ones. And depending on the availability of the API on a particular platform, the corresponding library can be supported. So, essentially, at this layer, we have at this layer of native library support, we have support for DirectX, OpenGL, Metal, Vulkan, and many such libraries.



So, to use Unity's default graphics APIs, Unity provides APIs, graphics APIs which in turn access the library APIs. Now, to use the default APIs, what a developer needs to do is to open the player settings editor interface. It can be accessed using this option: edit project settings, then select the player category, and then we can navigate to other settings and make sure the AutoGraphics API is checked or selected. When AutoGraphics API for a platform is checked, the player build



includes a set of built-in graphics APIs and uses the appropriate one at runtime to produce the best-case scenario. So, all the concerns related to implementations are taken care of by Unity. The developer does not need to bother about which API to use or what is supported for a given platform. Now, if this auto graphics API is not checked, then what happens? The editor uses the first API in the list, and if the default API is not supported by the specific platform, then Unity uses the next API in the AutoGraphics API list, and in this way, it continues until it finds the appropriate match. So, that is about library support. Let us also talk about some advanced rendering support. We have not discussed these before, but let us see how Unity supports those advanced features, rendering features, which make the experience more realistic and more usable. The first thing is physics.

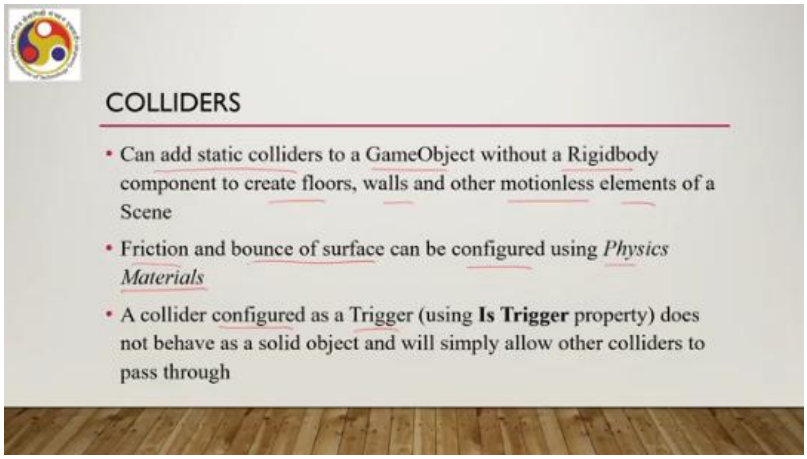
Now, Unity helps to simulate physics in your project. The application in your project ensures that the objects correctly accelerate and respond to collisions. So, essentially, the physical laws are supported. Now, Unity does this with the help of a component called the physics engine, which includes concepts of rigid bodies, colliders, joints, physics articulation, and character controllers. So, these are all components essential to give a realistic feeling.

What are rigid bodies? So, a rigid body is a concept. It is a property. It is a property that, when attached to a game object, allows the game object to respond to gravity. That means when it falls or when it bounces back from some surface, it appears to be more realistic,

in accordance with our everyday experience, rather than something fantastic or something imaginary. Of course, there are properties such as the isKinematic property. If we use this property, by using this property, we can remove the object from physics engine control and allow it to be moved kinematically from a script. That means it does not automatically follow the gravity property. So, using is kinematic, we can control the kinematics; we can allow it to follow gravity, we may disallow it to follow gravity; it is up to us.

So, such flexibility is provided by Unity. Then there is this colliders concept. It defines the shape of a game object for the purpose of physical collisions. These can be primitive colliders such as box collider, sphere collider, and capsule colliders, which are already there, or mesh collider, which is more processor-intensive but accurate. So, essentially, a collider allows you to deal with objects or object shapes after they collide or after they have a collision with some other objects.

We can add static colliders to a game object without a rigid body component to create floors, walls, and other motionless elements of a scene. Friction and bounce of a surface can be configured using physics materials. A collider configured as a trigger using the isTrigger property does not behave as a solid object and will simply allow other colliders to pass through. So, like rigid body, here also we can have flexibility whether to allow the physics laws to be governed, to be followed, or to have our own laws to be followed while creating a scene, while creating an experience.



And for that, several properties are already mentioned which we can make use of. Next is joints. It connects a rigid body to another rigid body or to a fixed point in space. Joints can apply forces that move the rigid bodies, and joint limits can restrict that movement. So, using the concept of a joint, we can actually control the movement of specific objects, specific characters, in a more realistic way.

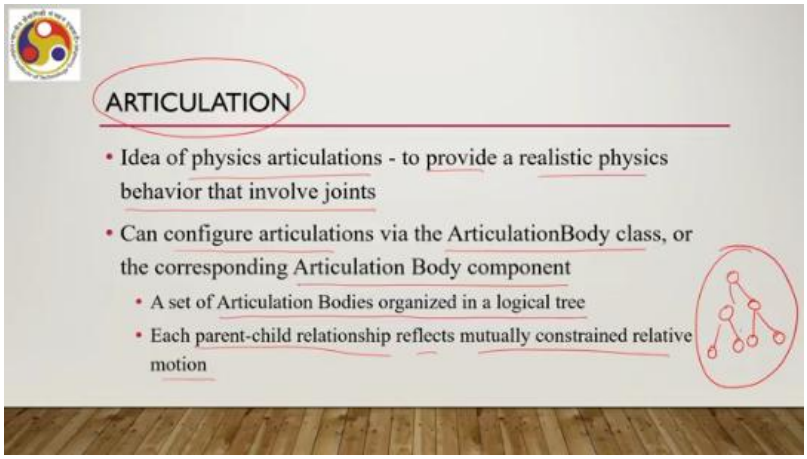
So, such joints can be useful to emulate a ball and socket joint like a hip or shoulder, any skeletal joint, doors and finger joints, or a piece of elastic. So, how they move overall, we can model that by using the concept of a joint. A related concept is articulation, essentially. The idea of physics articulation is implemented here.

which provides a realistic physics behavior that involves joints. So, when we have joints, then how the overall structure comprising joints, structure, or object moves, that is the articulation. We can configure articulations via the articulation body class or the corresponding articulation body component. So, here we do not need to go into the nitty-gritty of how to control each and every joint so that the overall articulation of the figure, or the shape, or the object appears to be realistic.

Instead, there is inbuilt support using which we can implement such constraints automatically. Now, such articulation body class essentially allows us to view the articulation bodies in the form of a logical tree. So, essentially this supports assuming such a logical tree representation for overall articulation implementation. Now, each parent-child relationship in that logical tree reflects mutually constrained relative motion.

So, essentially suppose we want to let So, this figure, suppose I want to make it move. So, each node is a joint, and the way the joints are related to the other joints and the way a joint can move with respect to the other joint, these things define the overall articulation. And

when you are dealing with such complex figures to move, and try to make those move, then you do not need to actually bother about the nitty-gritty of such joint movements and relative constraints.



Instead, this articulation body class or articulation body component can take care of it on your behalf. Also, Unity provides support for animation. Unity has a rich and sophisticated animation system. where you can get support for importing animation clips and or animation created within Unity. You can also have this humanoid animation retargeting support, which is essentially the ability to apply animations from one character model onto another.

Now, Unity provides support for a simplified workflow for aligning animation clips. convenient preview of animation clips, transitions and interactions between them, and management of complex interactions between animations with a visual programming tool. All these are supported by Unity, animating different body parts with different logics. So here again, animation is a quite complex task, and it goes through several stages like the 3D graphics pipeline stages we discussed earlier. But as a developer, you do not need to bother about those stages, those computations, or the implementation of those things.

Unity provides support for directly dealing with animations without going into the nitty-gritty of how to implement that animation pipeline. Also, if you recollect, we talked about engaging more senses to make the experience immersive. Now, so far while talking of rendering, it is about visual senses, but Unity also provides support for dealing with the audio sense, your ears. So, audio support is provided for an immersive experience. Unity has dedicated components for audio perception and playback.

The primary component is attached to a game object to make it play a sound. which is the audio source. This is the primary component attached to a game object to make it play a sound. Now, it allows you to play back an audio clip when triggered through a mixer, code, or by default. An audio clip can be any standard audio file such as MP3, WAV, and so on.

Audio listener is another component supported by Unity. It listens to all audio playing in the scene and transfers it to the computer speakers. It acts like ears in the game, internal ears in the game. All audio you hear is with respect to the positioning of this listener. And only one audio listener should be there in the scene by default, attached to the camera.

So you have audio source, audio listener, and also you have audio filters. Essentially, as the name suggests, such components output the audio source after some modification. So essentially, the output of the audio source or the intake of an audio listener can be modified with audio filters for various sound effects. Each filter comes with comes as its own component.

This is the internal organization of Unity, but the overall idea is that with filters you can modify the audio option you already have, and you can generate different effects, audio effects. So, let us summarize what we have covered so far. So, we talked about some useful features for the developer, which includes the implementation of various pipelines that Unity supports, different pipeline options, support for advanced graphics and audio, including physics support, audio support, and so on.

We will continue our discussion in the next part of this lecture, where we will talk about the scripting support and also we will talk about support for SDKs. So, I hope that you could follow the lecture, you could follow the flow and the topics. Looking forward to meeting you all again in the next part of this lecture. Thank you, goodbye, and happy learning.